

Reviewer Pack — Minimal Local Indeterminacy in Noncommutative Geometry and D...

Entry dfeffd1d401c · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-06-03 01:52:51 UTC

Minimal Local Indeterminacy in Noncommutative Geometry and DPLL Proof Depth

Entry ID: dfeffd1d401c **Verdict:** INCONCLUSIVE **Recorded:** 2026-06-03 01:52:51 UTC

1. Conjecture

Field A (mathematical branch): Noncommutative Geometry **Field B** (complexity object): DPLL Search Trees

Statement:

The minimal local indeterminacy of a noncommutative manifold associated with a given CNF is linearly correlated with the depth of the DPLL search tree for that CNF, such that the minimal local indeterminacy $I(M) = \Theta(\sqrt{d})$, where d is the depth of the DPLL search tree.

Rationale (proposer's reasoning):

Noncommutative geometry provides an abstract framework to study geometric properties of operators in algebraic structures, which can potentially reveal hidden structures in computational problems like SAT. The local indeterminacy measures the complexity of the operator's action on a manifold, and its correlation with the DPLL proof depth could expose novel connections between geometric complexity and computational complexity.

Taxonomy category: NoncommutativeGeometry (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): f7cf263871d92529

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

The conjecture is supported if the computed correlation coefficient between minimal local indeterminacy ($I(M)$) and DPLL search tree depth (d) across 30 seeds is ≥ 0.7 , with $I(M) \leq \sqrt{d} + 1$ for all seeds.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.70	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): -intitle:Minimal Local Indeterminacy AND Noncommutative Geometry AND DPLL Proof Depth - subject:Noncommutative Geometry AND DPLL Search Trees AND CNF - author:($I(M) = \Theta(\sqrt{d})$ OR minimal local indeterminacy) AND proof depth

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=1, elapsed=0.2s

5.1 Generated Python source

```
import random
import math
from itertools import combinations

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_cnf(n, m):
        variables = list(range(1, n + 1))
        clauses = []
        for _ in range(m):
            clause = [random.choice(variables) * (2 * random.randint(0, 1) - 1) for _ in range(n)]
            clauses.append(clause)
        return clauses

    def dpll(cnf):
        n = len(cnf)
        literals = list(range(1, n + 1)) + [-l for l in range(1, n + 1)]

        def solve(lits, cls):
            if not cls:
                return True
            pure_literal = next((l for l in lits if all(l not in c or -l not in c for c in cls)), None)
            if pure_literal is not None:
                new_lits = [l for l in lits if l != pure_literal and l != -pure_literal]
                return solve(new_lits, cls)
            unit_clause = next((c for c in cls if len(c) == 1), None)
            if unit_clause is not None:
                literal = unit_clause[0]
                new_cls = [c for c in cls if literal not in c and -literal not in c]
                return solve(lits, new_cls)
            p_literal = random.choice(literals)
            new_lits_true = lits + [p_literal]
```

```

        new_lits_false = lits + [-p_literal]
        return solve(new_lits_true, cls) or solve(new_lits_false, cls)

    return 1 if solve(literals, cnf) else 0

def local_indeterminacy(cnf):
    n = len(cnf)
    rank = 0
    for i in range(1, n + 1):
        for j in range(i + 1, n + 1):
            clause = [i, -j]
            if all(l not in c or -l not in c for c in cnf):
                rank += 1
    return rank

n_values = [5, 10, 15, 20, 30, 40]
results = []

for n in n_values:
    for _ in range(5): # Ensure at least 30 instances per seed
        cnf = generate_cnf(n, int(1.5 * n))
        depth = dpll(cnf)
        lcoh = local_indeterminacy(cnf)
        results.append((n, depth, lcoh))

if not results:
    return {
        "metric_name": "correlation_coefficient",
        "metric_value": None,
        "instances_tested": 0,
        "n_max": 0,
        "conjecture_holds": False,
        "counterexample": "mapping_undefined"
    }

n_values, depths, lcohs = zip(*results)
mean_depth = sum(depths) / len(depths)
mean_lcoh = sum(lcohs) / len(lcohs)
correlation_coefficient = (sum((d - mean_depth) * (1 - mean_lcoh) for d, l in zip(depths, lcohs))
                           / (math.sqrt(sum((d - mean_depth)**2 for d in depths)) *
                              sum((1 - mean_lcoh)**2 for l in lcohs)))

return {
    "metric_name": "correlation_coefficient",
    "metric_value": correlation_coefficient,
    "instances_tested": len(results),
    "n_max": max(n_values),
    "conjecture_holds": correlation_coefficient >= 0.7 and all(l <= math.sqrt(d) + 1 for d, l in zip(depths, lcohs)),
    "counterexample": ""
}

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [2**i - 1 for i in range(5, 30)]

    results = []

```

```

for seed in seeds:
    result = run_trial(seed)
    print(f"TRIAL: {{\"seed\": {seed}, \"metric_name\": \"{result['metric_name']}\", \"metric_value\": {result['metric_value']}\"}}")
    results.append(result)

mean_metric_value = sum(r[\"metric_value\"] for r in results if r[\"metric_value\"] is not None) / len(results)
support_fraction = sum(1 for r in results if r[\"conjecture_holds\"] ) / len(results)

if all(r[\"conjecture_holds\"] for r in results):
    print(f"RESULT: SUPPORTED mean={mean_metric_value} std=0.0 support_fraction={support_fraction}")
elif any(not r[\"conjecture_holds\"] for r in results) and any(r[\"metric_value\"] is not None for r in results):
    first_failing_seed = next(seed for seed, result in zip(seeds, results) if not result[\"conjecture_holds\"] and result[\"metric_value\"] is not None)
    print(f"RESULT: FALSIFIED counterexample=\"\" first_failing_seed={first_failing_seed}")
else:
    print("RESULT: INCONCLUSIVE reason=insufficient_data")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```

^^^^^^^^^^^^^^^^
File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_d66fca67.py", line 38, in solve
    return solve(new_lits, cls)
    ^^^^^^^^^^^^^^^^^^^^^^^^^
File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_d66fca67.py", line 47, in solve
    return solve(new_lits_true, cls) or solve(new_lits_false, cls)
    ^^^^^^^^^^^^^^^^^^^^^^^^^
File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_d66fca67.py", line 38, in solve
    return solve(new_lits, cls)
    ^^^^^^^^^^^^^^^^^^^^^^^^^
File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_d66fca67.py", line 47, in solve
    return solve(new_lits_true, cls) or solve(new_lits_false, cls)
    ^^^^^^^^^^^^^^^^^^^^^^^^^
File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_d66fca67.py", line 38, in solve
    return solve(new_lits, cls)
    ^^^^^^^^^^^^^^^^^^^^^^^^^
File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_d66fca67.py", line 47, in solve
    return solve(new_lits_true, cls) or solve(new_lits_false, cls)
    ^^^^^^^^^^^^^^^^^^^^^^^^^
File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_d66fca67.py", line 38, in solve
    return solve(new_lits, cls)
    ^^^^^^^^^^^^^^^^^^^^^^^^^
File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_d66fca67.py", line 47, in solve
    return solve(new_lits_true, cls) or solve(new_lits_false, cls)
    ^^^^^^^^^^^^^^^^^^^^^^^^^
File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_d66fca67.py", line 38, in solve
    return solve(new_lits, cls)
    ^^^^^^^^^^^^^^^^^^^^^^^^^
File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_d66fca67.py", line 47, in solve
    return solve(new_lits_true, cls) or solve(new_lits_false, cls)
    ^^^^^^^^^^^^^^^^^^^^^^^^^
File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_d66fca67.py", line 38, in solve
    return solve(new_lits, cls)
    ^^^^^^^^^^^^^^^^^^^^^^^^^
File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_d66fca67.py", line 47, in solve
    return solve(new_lits_true, cls) or solve(new_lits_false, cls)
    ^^^^^^^^^^^^^^^^^^^^^^^^^
File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_d66fca67.py", line 38, in solve
    return solve(new_lits, cls)
    ^^^^^^^^^^^^^^^^^^^^^^^^^

```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test code crashed before producing data, which means that the pre-registered support condition could not be evaluated. As a result, we cannot confirm the conjecture as supported. | next: Re-run the test with proper error handling to ensure it completes and produces the necessary data for correlation analysis.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	19560
2	preregistration	ollama_remote	glm4:latest	0	0	9301
3	novelty	ollama_remote	glm4:latest	0	0	10397
4	novelty	ollama_remote	glm4:latest	0	0	8977
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	19911
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	21473
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	19925
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	14560
9	judge	ollama_remote	glm4:latest	0	0	8891

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 132995 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in `research/audit/dfeffd1d401c.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/dfeffd1d401c.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/dfeffd1d401c.tar.gz` (if generated)